

	Bansilal Ramnath Agarwal Charitable Trust's Vishwakarma Institute of Information Technology Department of Artificial Intelligence and Data Science	
Student Name: Sathe Onkar Abasaheb		
Class: TY	Division: B	Roll No.: 372078
Semester: V		Academic Year: 2025-2026
Subject Name & Code: Cyber Security (AD31234B)		
Assignment No: 04 Title of Assignment: Compare symmetric and asymmetric key crypto system by referring to the appropriate use case with respect to the applicability and implement two algorithms of each type.		
Date of Performance: 18/08/2025		Date of Submission: 25/08/2025

ASSIGNMENT NO. 04

Aim:

Compare symmetric and asymmetric key cryptosystem by referring to the appropriate use case with respect to the applicability and implement two algorithms of each type.

Objective: The objective of this assignment is to compare symmetric and asymmetric key cryptosystems, referring to appropriate use cases, and implement two algorithms of each type.

Materials Required:

Computer with Python programming environment.

Procedure:

1. Introduction

a. Overview of Symmetric and Asymmetric Key Cryptosystems

Cryptography is the science of securing communication and data through encryption. Two fundamental types of cryptographic systems are **symmetric key cryptosystems** and **asymmetric key cryptosystems**.

Symmetric Key Cryptosystems use a single key for both encryption and decryption. The sender and receiver share the same secret key, which must be kept confidential from any third party. Common symmetric algorithms include AES (Advanced Encryption Standard), DES (Data Encryption Standard), Blowfish, and Triple DES.

Asymmetric Key Cryptosystems, also called public-key cryptosystems, use a pair of mathematically linked keys: a **public key** and a **private key**. The public key is shared openly and used for encryption, while the private key is kept secret and used for decryption. Examples include RSA (Rivest-Shamir-Adleman), ElGamal, and ECC (Elliptic Curve Cryptography).

b. Differences between Symmetric and Asymmetric Encryption Algorithms

Feature	Symmetric Key Cryptosystem	Asymmetric Key Cryptosystem
Number of keys	Single shared secret key	Key pair: public key (encrypt), private key (decrypt)
Speed	Generally faster due to simpler algorithms	Slower due to complex mathematical operations
Key distribution	Requires secure key exchange channels	Public key can be distributed openly
Use cases	Bulk data encryption, real-time encryption	Secure key exchange, digital signatures, email encryption
Security dependency	Key secrecy and secure key exchange	Security relies on the difficulty of solving mathematical problems like factoring large numbers or discrete logarithms

c. Strengths and Limitations of Each Type

Symmetric Key Cryptosystems:

Strengths: Efficient and fast, suitable for encrypting large amounts of data.

Limitations: Key distribution problem; both parties must securely exchange and manage the same secret key, which can be a risk.

Asymmetric Key Cryptosystems:

Strengths: Simplifies key distribution by allowing public sharing of the encryption key. Provides features like digital signatures and non-repudiation.

Limitations: Computationally intensive, slower than symmetric cryptography, and usually less efficient for encrypting large data volumes.

2. Use Case Comparison

a. Use Case for Symmetric Key Cryptosystem

Use Case: Encrypting data stored on a local hard drive.

In this scenario, a symmetric key cryptosystem like AES is appropriate because the data owner encrypts and decrypts the data on the same device or within a trusted environment.

The symmetric key can be securely stored or derived from user credentials.

The speed and efficiency of symmetric encryption ensure minimal performance impact.

b. Use Case for Asymmetric Key Cryptosystem

Use Case: Secure email communication between two users.

Asymmetric cryptosystems like RSA enable one user to encrypt a message with the recipient's public key, ensuring only the recipient can decrypt it with their private key.

This eliminates the need for a prior secure exchange of secret keys.

Additionally, digital signatures can verify the sender's identity.

3. Symmetric Key Cryptosystem

a. Selected Algorithms: AES and DES

AES is a widely used symmetric encryption standard with variable key sizes (128, 192, 256 bits) and strong security.

DES is an older standard using a 56-bit key; it is now largely considered insecure but serves educational purposes.

b. Implementation Overview

Encryption steps involve plaintext padding (if required), key scheduling, and block cipher operations.

Decryption reverses the process to retrieve the original plaintext.

Python libraries like `pycryptodome` or `cryptography` provide APIs for these algorithms.

c. Security and Efficiency

AES offers strong security against modern cryptanalysis with high throughput, making it ideal for real-time data encryption.

DES is vulnerable to brute-force attacks due to its short key length and is thus obsolete for secure applications.

4. Asymmetric Key Cryptosystem

a. Selected Algorithms: RSA and ElGamal

RSA is based on the difficulty of factoring large composite numbers.

ElGamal relies on the discrete logarithm problem and provides semantic security.

b. Implementation Overview

Key generation involves generating large primes and computing related parameters.

Encryption uses the recipient's public key, while decryption uses the private key.

Python libraries such as `pycryptodome` or custom implementations using `sympy` and `random` modules can be used.

c. Security and Computational Complexity

RSA's security depends on key size; larger keys offer stronger security but increase computation time.

ElGamal provides strong security guarantees but is computationally more expensive than RSA.

Both algorithms are inefficient for encrypting large data, hence often used for encrypting small data blocks or keys.

5. Comparison and Contrast

Aspect	Symmetric Key Cryptosystems	Asymmetric Key Cryptosystems
Security Properties	Relies on key secrecy; vulnerable to key leakage	Based on mathematical hardness assumptions; supports digital signatures

Aspect	Symmetric Key Cryptosystems	Asymmetric Key Cryptosystems
Key Management	Difficult due to secure key exchange	Easier due to public key distribution
Computational Complexity	Low, fast encryption/decryption	High, slower encryption/decryption
Suitability	Bulk data encryption, real-time applications	Secure key exchange, authentication, digital signatures
Example Use Case	Disk encryption, VPN	Email encryption, SSL/TLS handshake

Input/Output :

1.AES:

```

✓ [1] !pip install pycryptodome
0s

Collecting pycryptodome
  Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
  Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 23.9 MB/s eta 0:00:00
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

def aes_encrypt(plaintext, key):
    cipher = AES.new(key, AES.MODE_CBC)
    ct_bytes = cipher.encrypt(pad(plaintext.encode(), AES.block_size))
    return cipher.iv, ct_bytes

def aes_decrypt(iv, ciphertext, key):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    pt = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return pt.decode()

# Test AES
key = get_random_bytes(16) # AES-128
plaintext = "Hello, I am sanket mahajan."
iv, ciphertext = aes_encrypt(plaintext, key)
decrypted_text = aes_decrypt(iv, ciphertext, key)

print("AES Encryption")
print("Original:", plaintext)
print("Ciphertext (hex):", ciphertext.hex())
print("Decrypted:", decrypted_text)
print()

AES Encryption
Original: Hello, I am sanket mahajan.
Ciphertext (hex): edad017c2972f81d73904f916dc22fcc8845ef8a0b83ac249d456155ccba976d
Decrypted: Hello, I am sanket mahajan.

```

2.RSA:

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

def rsa_keygen():
    key = RSA.generate(2048)
    private_key = key.export_key()
    public_key = key.publickey().export_key()
    return private_key, public_key

def rsa_encrypt(public_key, plaintext):
    recipient_key = RSA.import_key(public_key)
    cipher = PKCS1_OAEP.new(recipient_key)
    encrypted = cipher.encrypt(plaintext.encode())
    return encrypted

def rsa_decrypt(private_key, ciphertext):
    key = RSA.import_key(private_key)
    cipher = PKCS1_OAEP.new(key)
    decrypted = cipher.decrypt(ciphertext)
    return decrypted.decode()

# Test RSA
private_key, public_key = rsa_keygen()
plaintext = "Hey Sanket Mahajan."
ciphertext = rsa_encrypt(public_key, plaintext)
decrypted_text = rsa_decrypt(private_key, ciphertext)

print("RSA Encryption")
print("Original:", plaintext)
print("Ciphertext (hex):", ciphertext.hex())
print("Decrypted:", decrypted_text)
```

RSA Encryption
Original: Hey Sanket Mahajan.
Ciphertext (hex): 7d62b58e2aa3b309ded37fa1cbb0445edb1409dc217bcd54d3b23c42af938a49bfff81387aefd3fb30b664353dd17a9bd952d2c06012e6db3cc2466e15938
Decrypted: Hey Sanket Mahajan.

Conclusion:

This assignment highlights the fundamental differences between symmetric and asymmetric key cryptosystems. Symmetric algorithms like AES offer efficient encryption suited for large data volumes but suffer from key distribution challenges. Asymmetric algorithms like RSA solve the key distribution problem and provide additional security features at the cost of computational complexity.

Understanding these systems' appropriate use cases is crucial for designing secure communication protocols. Proper key management remains the backbone of any cryptographic system's security. As cryptography advances, hybrid systems combining symmetric and asymmetric methods (e.g., using asymmetric keys to exchange symmetric session keys) are prevalent, balancing security and efficiency.